

TNPSC Combined Technical Services

Paper Code 554 — Unit I

PROGRAMMING IN C, PYTHON AND OBJECT ORIENTED PROGRAMMING

Expanded textbook-style study notes

These notes explain the complete Unit I syllabus continuously, topic by topic, from fundamentals to implementation concepts. The aim is to provide a self-contained study text rather than short revision points.

PART I — C PROGRAMMING

Data Types

A data type tells the C compiler what kind of value a variable can store, how much memory is normally required, how the stored bits are interpreted, and which operations are meaningful on that value. C is a statically typed language: a variable has a declared type, and the compiler uses that type when checking expressions and generating machine code. The fundamental arithmetic types include char, integer types, and floating-point types. Integer types include short, int, long and long long, each of which may also be signed or unsigned. Floating-point types include float, double and long double. The exact size of several types is implementation-dependent, so portable C programs should use sizeof rather than assuming a universal byte size. The char type represents a character-sized integer type and can participate in arithmetic. signed and unsigned affect the range of representable integer values. Unsigned types represent only non-negative values, giving them a larger positive range for the same storage width. C also provides derived and user-defined types. Arrays contain elements of one type; pointers store addresses; structures group members of different types; unions allow different members to share the same storage; enumerations define named integral constants; typedef creates an alternative type name.

- sizeof(type) or sizeof expression gives the storage size in bytes as a size_t value.
- Integer conversion and usual arithmetic conversions affect mixed-type expressions.
- Signed overflow is not defined as ordinary wraparound in C; unsigned arithmetic is modulo 2^N for the relevant width.
- const qualifies an object so that modification through that designation is not permitted.

Expressions

An expression is a combination of operands, operators, function calls, constants and variables that produces a value or otherwise performs an operation. C has arithmetic, relational, logical, bitwise, assignment, conditional, increment/decrement and other operators. Arithmetic operators include +, -, *, / and %. Integer division discards the fractional part. The remainder operator applies to integer operands. Relational operators (<, <=, >, >=, ==, !=) produce an integer result of 0 or 1. Logical operators &&, || and ! operate on truth values, where zero is false and non-zero is true. Operator precedence determines which operators bind more strongly, while associativity determines evaluation grouping when operators have equal precedence. Parentheses should be used when the intended grouping is important. A crucial distinction is that precedence determines grouping of an expression; it does not mean that every operand is evaluated in a simple left-to-right order. The conditional operator ?: is the only ternary operator in C. It selects one of two expressions based on a condition. Assignment operators include =, +=, -=, *=, /=, %= and corresponding bitwise forms.

- x++ and ++x both increment x but differ in the value of the expression.
- && and || use short-circuit evaluation: the second operand may not be evaluated.
- == tests equality; = performs assignment.
- Bitwise operators &, |, ^ and ~ operate on integer representations.

Input and Output Operations

C's standard input/output facilities are supplied mainly through stdio.h. printf writes formatted output to a stream, commonly stdout. scanf reads formatted input from stdin. getchar reads a character and putchar writes one character. fgets reads a line or character sequence into a buffer with a size limit; puts writes a string followed by a newline. Format specifiers tell formatted I/O how to interpret arguments. For example, %d is commonly used for int, %f for floating output and %lf for double input in scanf. For scanf, scalar variables generally need their addresses because the function must modify them; hence &x; is commonly supplied. Character arrays used as strings are passed as pointers to their first element and do not use & for the array name in the ordinary call. Whitespace handling is important. scanf("%d",&x;) leaves the following newline in the input stream in many situations, which can affect subsequent character input. fgets is often preferable when reading complete lines because it can retain spaces and allows a maximum size to be specified.

- Always match a format specifier with the correct argument type.

- For strings, leave space for the terminating '\0'.
- Use a field width or size limit when appropriate to avoid writing beyond a buffer.
- EOF is a special indication used by input functions to represent end-of-file or an input error in relevant APIs.

Decision Making and Branching Statements

Decision statements allow a program to choose different execution paths. The if statement evaluates a controlling expression: zero is false and non-zero is true. if may be followed by else, and multiple conditions can be represented using else-if chains. The switch statement selects among cases based on an integral or enumeration controlling expression. Each case labels a possible value. Unless execution is deliberately terminated, control can fall through from one case to the next. break is therefore frequently used to prevent unintended fall-through. A default label handles values for which no case matches. Nested conditional statements are possible, but excessive nesting can reduce readability. The conditional operator can express a small value selection compactly, while if/else is usually clearer for multi-step actions.

- if tests truth; C has no separate Boolean syntax requirement in traditional C, although provides bool in modern C.
- switch case labels must be integer constant expressions.
- break exits the nearest enclosing switch or loop.
- default is optional.

Looping Statements

Loops repeat a statement or block while a condition or iteration rule permits. C provides while, do-while and for loops. A while loop tests before each iteration, so it may execute zero times. A do-while tests after the body and therefore executes at least once. The for loop has initialization, condition and iteration expressions. Any of these may be omitted, so for(;;) creates an infinite loop unless control leaves it. Nested loops are useful for matrices and other multidimensional data but increase the running time according to the product of their iteration counts. break terminates the nearest loop or switch. continue skips the remaining body of the current iteration and begins the next iteration according to the loop's semantics. Careful control of loop variables and termination conditions is essential to avoid infinite loops and off-by-one errors.

- while: test first; do-while: body first.
- for is especially useful when the iteration variable follows a predictable progression.
- Nested loops commonly lead to $O(n^2)$, $O(n^3)$, etc., depending on their bounds.

Arrays — Declaration, Initialization and One-Dimensional Arrays

An array is a fixed-size sequence of objects of the same type stored contiguously. C array indexing begins at zero. For an array declared as `int a[5]`, valid indices are 0 through 4. Accessing outside the valid range produces undefined behaviour; C does not automatically perform bounds checking. An array declaration specifies the element type and number of elements. Initialization can be supplied at the point of definition, for example `int a[5]={1,2,3};`. The remaining elements are initialized to zero. An array can also be initialized with fewer explicitly listed elements, while a character array initialized from a string literal includes a terminating null character if there is room. Because elements are contiguous, the address of an element can be computed using its base address and index. This makes random access $O(1)$. Insertion or deletion in the middle of a normal fixed array requires shifting elements when maintaining a logical sequence.

- `a[i]` is equivalent to `*(a+i)` for an ordinary array expression.
- Array size in a declaration is not the same thing as the number of initialized elements.
- `sizeof(array)` inside the same scope as the actual array gives total storage, not number of elements.
- When an array is passed to a function parameter, it is adjusted to a pointer parameter.

Two-Dimensional Arrays

A two-dimensional array is an array whose elements are themselves arrays. It is commonly used for tables and matrices. For `int a[3][4]`, there are 3 rows and 4 columns, and C stores the elements in row-major order: all elements of one row are followed by those of the next row. Nested loops are normally used for traversal. The first index selects the row and the second selects the column. Matrix addition requires corresponding dimensions and adds each pair of elements. Matrix multiplication requires the number of columns of the first matrix to equal the number of rows of the second; each result element is the dot product of one row and one column. When a multidimensional array is passed to a function, the compiler needs enough dimension information to calculate element addresses. For a normal two-dimensional parameter, the second dimension is therefore important.

- `a[i][j]` identifies row `i` and column `j`.
- Matrix multiplication is typically $O(r_1 \times c_1 \times c_2)$.
- Row-major layout is a property of C's multidimensional array representation.

Strings and String Operations

C does not have a built-in string object type. A string is conventionally represented by an array of `char` terminated by the null character `'\0'`. A string literal such as `"hello"` contains the characters `h`, `e`, `l`, `l`, `o` and a terminating null character. The string library in `string.h` provides functions such as `strlen`, `strcpy`, `strncpy`, `strcat`, `strncat`, `strcmp` and related operations. `strlen` counts characters before the null terminator. `strcpy` copies a source string including its terminator, assuming the destination has sufficient capacity. `strcat` appends a source string to the destination, again requiring sufficient space. `strcmp` compares two strings lexicographically and returns zero when they are equal; a negative or positive result indicates ordering but programs should generally test `<0`, `==0` or `>0` rather than depend on a particular numeric value.

- A C string must have a terminating `'\0'` for functions that expect a null-terminated string.
- `strlen` does not count `'\0'`.
- `strcpy/strcat` require enough destination storage; misuse can cause buffer overflow.
- `strcmp` compares character sequences, not pointer addresses.

String Arrays

A collection of strings can be represented in several ways. A two-dimensional character array such as `char names[5][20]` reserves storage for five strings of up to 19 characters plus the terminator. An array of pointers such as `char *names[5]` stores five pointers, and each pointer can refer to a separate character sequence. The two approaches differ in memory layout and mutability. A two-dimensional array allocates fixed-size slots contiguously. An array of pointers can point to strings of different lengths and can also point to dynamically allocated storage. String literals should not be modified through pointers intended to point to them.

- `char s[5][20]` gives five fixed-capacity string buffers.
- `char *s[5]` gives five pointer objects; the pointed-to storage is separate.
- Always ensure the destination capacity is sufficient for copying.

Simple Programs, Sorting, Searching and Matrix Operations

Many elementary C programs combine the preceding constructs. Typical tasks include finding maximum/minimum, computing sums and averages, counting occurrences, reversing data, checking prime numbers, generating sequences, sorting arrays, searching for values and performing matrix operations. Sorting rearranges elements according to an ordering. A simple bubble-sort implementation repeatedly compares adjacent elements and swaps them if they are out of order. Selection sort repeatedly selects an extreme element and places it in the next position. Insertion sort grows a sorted prefix and inserts each new value into the correct position. Searching locates an element. Linear search examines entries one by one and works without sorted input. Binary search requires sorted data and repeatedly halves the search interval, giving logarithmic search time. Matrix programs commonly implement addition, transpose, multiplication, diagonal operations and row/column calculations using nested loops.

- When writing algorithms, distinguish an element's value from its index.

- Binary search is valid only when the ordering invariant is maintained.
- Matrix dimensions must be checked before operations such as addition and multiplication.

Functions — Definition and Declaration

A function is a named unit of computation that can receive arguments and return a value. A function declaration, also called a prototype, tells the compiler the function's name, return type and parameter types before a call is compiled. The definition supplies the function body. A function improves modularity, reuse and readability. C passes arguments by value: the called function receives copies of the argument values. If a function must modify an object in the caller, it can receive a pointer to that object and dereference the pointer. This is commonly described as achieving pass-by-reference-like behaviour, but technically C itself uses pass-by-value for function arguments, including pointer values. A function can have no parameters, parameters of different types, or a pointer/array parameter. A return statement supplies the function's result when its return type is not void.

- Declaration and definition are different concepts.
- Function prototype enables type checking at the call site.
- C has pass-by-value; pointer arguments allow caller objects to be modified.
- void means no value is returned from the function.

Pass by Value and Pointer-Based Reference Semantics

In pass by value, the value of an argument is copied into the corresponding parameter. Changing the parameter does not change the original object. For example, a function receiving `int x` cannot change the caller's `int` merely by assigning to `x`. To modify the caller's object, pass its address. A function receiving `int *p` can execute `*p=...` and thereby change the object to which `p` points. The pointer itself is still passed by value: the called function receives a copy of the address. For multiple outputs, pointers are commonly used as output parameters. Arrays naturally interact with this model because an array expression used as a function argument generally becomes a pointer to its first element.

- `int f(int x)` cannot directly modify caller's integer through `x`.
- `int f(int *p)` can modify the caller's integer using `*p`.
- Passing a pointer is not the same language mechanism as true reference parameters in C++.

Recursion

Recursion occurs when a function directly or indirectly calls itself. A recursive solution needs a base case that stops recursion and a recursive case that reduces the problem toward the base case. Without a terminating condition, calls continue until resources are exhausted. Each recursive call creates a new activation record containing parameters, local variables and return information. Therefore recursion consumes call-stack space. Examples include factorial, Fibonacci, tree traversal, binary search and divide-and-conquer algorithms. A recursive algorithm can sometimes be transformed into iteration. Recursion is particularly natural for hierarchical structures such as trees because a subtree is itself a tree.

- Base case prevents infinite recursion.
- Recursive call must progress toward a base case.
- Recursion depth affects stack usage.
- Naive recursive Fibonacci has exponential-time growth because subproblems are repeatedly recomputed.

Pointers — Definition and Initialization

A pointer is an object whose value is an address or a null pointer value. If `int x=10;` then `int *p=&x;` declares `p` as a pointer to `int` and initializes it with the address of `x`. Dereferencing `p` with `*p` accesses the pointed-to `int`. The address-of operator `&` produces an address. The indirection operator `*` dereferences a pointer in an appropriate expression. A null pointer represents that the pointer does not point to a valid object or function. It should not be dereferenced. Pointers are central to dynamic memory allocation, arrays, strings, structures, linked data structures and functions that modify

caller-owned objects.

- `int *p` means `p` points to `int`.
- `&x`; obtains `x`'s address.
- `*p` accesses the object pointed to by `p`.
- An uninitialized pointer contains an indeterminate value and must not be dereferenced.
- `NULL` is a traditional null-pointer macro; modern C also provides `nullptr` in newer language revisions/implementations depending on standard support.

Pointer Arithmetic

Pointer arithmetic is defined in terms of elements of the pointed-to type. If `p` points to an element of an array, `p+1` points to the next element, not merely one byte after `p`. The compiler scales the address adjustment by `sizeof(*p)`. Subtracting two pointers into the same array (or one past its end) gives the number of elements between them. Comparing pointers has defined meaning in specific array/object relationships. Pointer arithmetic outside the relevant array object is not generally valid. Pointer arithmetic explains the close relationship between arrays and pointers. For an array `a`, `a+i` designates the address of element `i` in ordinary expressions, and `*(a+i)` accesses that element.

- `p+1` advances by `sizeof(*p)` bytes conceptually.
- Pointer subtraction yields an element distance when the pointers refer into the same array object.
- One-past-the-end pointers may be formed for traversal but must not be dereferenced.

Pointers and Arrays

Arrays and pointers are closely related but are not identical types. In most expressions, an array name is converted to a pointer to its first element. This is why `a[i]` and `*(a+i)` are equivalent. However, `sizeof(array)` and `&array` have different meanings from their pointer counterparts. When an array is passed to a function, the parameter is adjusted to a pointer. Thus a function receiving `int a[]` does not receive a complete array object and cannot determine its length from `sizeof(a)` inside the function. The length is normally supplied separately. Pointers can also be used to traverse arrays. Incrementing an `int` pointer moves from one integer to the next. For multidimensional arrays, pointer types must preserve the correct row width for address calculations.

- `array` $\neq$ `pointer` even though arrays often decay to pointers.
- `sizeof(a)` differs from `sizeof(pointer)` when `a` is an actual array.
- Pass array length explicitly to functions.

Structures — Definition and Declaration

A structure is a user-defined type that groups related objects, potentially of different types, under one name. The members have separate storage within the structure, subject to alignment and padding. A structure type can be declared with `struct` followed by a tag, and objects of that type can then be created. Members are accessed with the dot operator when the object itself is a structure. If `p` is a pointer to a structure, `p->member` is shorthand for `(*p).member`. Structures can contain arrays, pointers, other structures and many other types. Structures are fundamental for records such as student information, employee data, coordinates and linked-list nodes.

- `struct` groups heterogeneous members.
- Use `.` for a structure object and `->` for a pointer to structure.
- Padding may make `sizeof(struct)` larger than the sum of member sizes.
- A structure can contain a pointer to its own type, enabling linked structures.

Structure Within a Structure

A structure may contain another structure as a member. This models hierarchical records naturally. For example, an Employee can contain an Address structure. Access proceeds through the outer member and then the inner member,

such as `employee.address.pin`. Nested structures are different from a pointer-based relationship: if the inner structure is stored directly as a member, its storage is part of the outer structure. A pointer member instead stores only an address and refers to an object elsewhere.

- Direct nested structure → inner object's storage is part of outer object.
- Pointer to structure → separate object can be referenced.
- Nested structures improve organization of related data.

Unions

A union is a user-defined type whose members share the same storage location. At a given time, the stored representation is intended to be interpreted through the member that was most recently written, subject to C's detailed rules. The size of a union is sufficient for its largest member, with alignment considerations. Unlike a structure, where each member generally has its own storage, union members overlap. This makes unions useful when an object may hold one of several alternative representations and memory efficiency matters. Writing one member and reading a different member requires care because interpretation rules are subtle and can depend on the exact types and operation. Unions should therefore be used deliberately rather than as a generic type-punning mechanism.

- Union members overlap in storage.
- `sizeof(union)` is at least large enough for its largest member.
- Only one alternative representation is normally considered active at a time.
- Structure and union differ primarily in member storage arrangement.

Programs Using Structures and Unions

Structure-based programs commonly create arrays of records, search records by a key, sort records, pass structures to functions and maintain linked data structures. A linked-list node, for example, commonly contains a data field and a pointer to another node of the same structure type. Union-based programs are useful for variant records where a discriminator identifies which member should be interpreted. A common design uses an enum or integer tag plus a union containing the possible payloads. The tag and union together form a safer variant representation than guessing which member is currently meaningful.

- Use a discriminator/tag when a union can represent multiple logical variants.
- Structures can be passed by value or through pointers.
- Self-referential structures use pointers to the same structure type.

Storage Classes

Storage-class concepts describe aspects such as scope, linkage and storage duration. The traditional C keywords include `auto`, `register`, `static` and `extern`, although modern C specifications describe these concepts with more nuance. An automatic local object normally has automatic storage duration and is created when execution enters its block and destroyed when the block exits. `static` applied to a local object gives it static storage duration: its value persists between function calls, while its block scope remains. A file-scope static declaration gives internal linkage, making the identifier usable only within that translation unit. `extern` declares an identifier whose definition is supplied elsewhere, commonly used to share global objects across source files. `register` historically suggested that a variable be kept in a CPU register; modern compilers generally make their own optimization decisions, and taking the address of a register-declared object is restricted by the language.

- `static local` → persists between calls.
- `file-scope static` → internal linkage.
- `extern` → refers to an externally defined object/function.
- `auto` is the default storage duration for ordinary block-scope local variables.

Preprocessor Directives

The C preprocessor runs before the main compilation phase. It handles directives beginning with #, including #include, #define, #if, #ifdef, #ifndef, #elif, #else and #endif. #include inserts the contents of a header into the preprocessing stream. #define creates macros. Function-like macros can accept parameters, but parentheses should be used carefully to avoid precedence surprises. Conditional compilation can select code depending on macros or compilation conditions. Header guards commonly use #ifndef, #define and #endif to prevent a header's contents from being processed multiple times within a translation unit. Modern compilers may also support #pragma once, but traditional include guards are portable.

- Preprocessing occurs before compilation.
- #define performs textual macro substitution rather than type-checked function semantics.
- Use parentheses in macro definitions such as #define SQR(x) ((x)*(x)).
- Conditional compilation can include or exclude source regions.

File Handling

C file handling is provided through stdio.h and the FILE type. fopen opens a file and returns a FILE pointer or NULL on failure. fclose closes an opened stream. Common modes include r for reading, w for writing/truncating, a for appending, and corresponding + forms for update access; b may be used for binary mode where relevant. Character and line operations include fgetc, fputc, fgets and fputs. Formatted operations include fprintf and fscanf. Binary block operations include fread and fwrite. The file-position functions fseek, ftell and rewind support navigation and position queries. Text and binary representations should not be confused. Portable binary serialization of arbitrary C structures can be complicated by padding, alignment, endianness and representation differences. Therefore simply fwrite-ing a structure is not necessarily a portable file format.

- Always check whether fopen returned NULL.
- Close files with fclose when finished.
- EOF is used by many character-input routines.
- w normally truncates an existing file.
- a writes at the end; seeking behaviour for append has special semantics.

PART II — PYTHON PROGRAMMING

Python Interpreter and Interactive Mode

Python is an interpreted, high-level programming language with dynamic typing and automatic memory management. In the usual CPython implementation, source code is compiled to bytecode and executed by a virtual machine; therefore the simple statement that Python is 'only interpreted' is an oversimplification. The Python interpreter can run a script file or an interactive read-evaluate-print loop (REPL). Interactive mode is useful for experimenting with expressions, inspecting objects and testing small pieces of code. A statement entered at the prompt is parsed and executed immediately. Python uses indentation to delimit blocks instead of braces. Indentation is syntactically significant, so consistent indentation is required. Comments begin with # outside strings.

- Interactive mode is commonly called the REPL.
- Indentation is part of Python syntax.
- Python is dynamically typed: names are bound to objects and do not require a fixed declared type in the usual syntax.

Data Types

Python's built-in data model includes numeric types such as int, float and complex; Boolean values bool; the singleton None; sequences such as str, list and tuple; mappings such as dict; and set types. Python variables are names bound to objects rather than fixed typed memory locations in the C sense. Integers have arbitrary precision in normal Python implementations, so integer arithmetic can grow beyond fixed-width machine integer ranges, subject to available memory. float normally represents double-precision floating-point values in CPython. Complex numbers contain real and imaginary components. Objects also have identity, type and value. The type() function reports an object's type. isinstance() checks whether an object is an instance of a class or compatible class relationship.

- int is arbitrary precision in ordinary Python.
- bool is a subclass of int conceptually in Python's type hierarchy.
- None represents absence of a value and is a singleton.
- Mutable objects can be changed after creation; immutable objects cannot be changed in place.

Statements

A statement performs an action, such as assignment, conditional branching, looping, function definition, import or exception handling. Simple statements can appear on one logical line, while compound statements introduce an indented suite. Assignment binds names to objects. Multiple assignment can unpack iterables, for example a,b = 1,2. Augmented assignment operators such as += combine an operation with assignment, but their exact behaviour depends on the object's implementation. Control-flow statements include if/elif/else, for, while, break, continue and pass. The pass statement does nothing and is useful as a placeholder where syntax requires a statement.

- Python uses indentation to define suites.
- for iterates over an iterable, not necessarily over integer indices.
- break exits the nearest loop; continue skips to the next iteration.

Expressions, Boolean Values and Operators

Expressions produce values. Python has arithmetic operators (+, -, *, /, //, %, **), comparisons, Boolean operators and, or, not, bitwise operators, membership operators in and not in, identity operators is and is not, indexing and function calls. Python's Boolean values are True and False. In conditional contexts, many objects have truth values: None, False, numeric zero and empty collections are falsey, while most other objects are truthy. The operators and and or short-circuit and return one of their operands rather than necessarily a Boolean object. This behaviour is frequently used for conditional value selection. == compares values according to the objects' equality semantics, while is tests object identity and is normally used for singleton checks such as x is None.

- / performs true division; // performs floor division.

- `**` is exponentiation.
- `==` is equality; `is` is identity.
- `and/or` short-circuit.
- `not` has Boolean-result semantics.

Strings

A Python string is an immutable sequence of Unicode characters. Literals can use single, double, triple-quoted or other supported forms. Because strings are immutable, operations that appear to modify a string actually create another string object. Indexing retrieves one-character strings; slicing retrieves subsequences. Common methods include `lower`, `upper`, `strip`, `split`, `join`, `replace`, `startswith`, `endswith` and `find`. Formatting can be performed with f-strings, `format()` and other mechanisms. Unicode support means that Python strings represent text at a higher abstraction level than raw byte arrays. `bytes` is a separate immutable sequence type used for binary data. Encoding converts text to bytes; decoding converts bytes to text.

- Strings are immutable.
- Indexing starts at zero.
- Slicing uses `start:stop:step` and excludes `stop`.
- `join` is called on the separator string and receives an iterable of strings.

Arrays of Numbers

Python lists can store numbers and provide general-purpose dynamic sequences, but they store references to Python objects and therefore are not equivalent to a C contiguous numeric array. For numerical storage, Python also provides the `array` module, which stores values using a constrained type code, and external scientific libraries such as NumPy provide dense multidimensional arrays. For the syllabus concept of arrays of numbers, understand the distinction between a general list and a typed contiguous numeric representation. General lists support mixed types, while typed numeric arrays impose representation constraints and can be more memory-efficient for homogeneous data.

- list is a general mutable sequence.
- `array.array` stores values using a specified type code.
- A Python list is not the same memory model as a C array.

Lists

A list is an ordered, mutable sequence. It can contain objects of different types and can grow or shrink dynamically. Common operations include indexing, slicing, `append`, `extend`, `insert`, `remove`, `pop`, `clear`, `sort` and `reverse`. `append` adds one object at the end; `extend` adds elements from an iterable. `insert` places an element at a specified position and may require shifting references. `pop` removes and returns an element, with `pop()` normally removing the last item. `remove` searches for and removes the first matching value. List comprehensions provide a compact syntax for constructing lists from iterables with optional conditions. They create a new list and are useful for transformations and filtering.

- `append` adds one element; `extend` adds many elements from an iterable.
- Lists are mutable.
- Indexing is $O(1)$ in the common implementation; insertion near the front is $O(n)$.
- `sort` modifies the list; `sorted()` returns a new sorted list.

Tuples

A tuple is an ordered, immutable sequence. Tuples are created using comma-separated values, with parentheses often used for readability. A one-element tuple requires a trailing comma, such as `(5,)`. Tuples support indexing, slicing, iteration, membership and unpacking. Because the tuple container is immutable, its references cannot be replaced, appended to or deleted. However, if a tuple contains a mutable object, that inner object may itself be modified. Tuples

are commonly used for fixed collections of related values, function return values and unpacking. They can be hashable when all their elements are hashable, allowing suitable tuples to serve as dictionary keys or set members.

- Immutable container.
- One-element tuple: (x,).
- Packing and unpacking are important tuple operations.
- Tuple immutability does not make contained mutable objects immutable.

Dictionaries

A dictionary is a mutable mapping from hashable keys to values. Keys are unique within a dictionary. Modern Python dictionaries preserve insertion order as a language guarantee, but their fundamental abstraction remains key-to-value mapping. Operations include creating entries, retrieving values, updating values, deleting entries and iterating over keys, values or key-value pairs. `get()` can retrieve a key with a default instead of raising `KeyError`. `keys()`, `values()` and `items()` provide dynamic view objects. Dictionary lookup is expected $O(1)$ on average under normal hashing assumptions, though pathological collisions can affect performance. Keys must be hashable and have suitable equality/hash behaviour.

- Keys must be hashable.
- Duplicate assignment to the same key replaces the old value.
- `get()` avoids `KeyError` when a default is supplied.
- `items()` yields key-value pairs.

Functions

Functions are defined using `def` followed by a name, parameter list and indented body. Python functions are first-class objects: they can be assigned to names, stored in collections, passed as arguments and returned from other functions. Parameters can be positional, keyword, default-valued, positional-only or keyword-only depending on syntax. `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dictionary. Python uses object-reference/value semantics: arguments are passed as object references by assignment. If a mutable object is modified in place inside a function, the caller can observe that mutation. Rebinding the local parameter name does not rebinding the caller's name. A function can return multiple values syntactically by returning a tuple. If no explicit return value is supplied, the function returns `None`.

- Functions are first-class objects.
- Default argument expressions are evaluated when the function is defined, not on every call.
- Avoid mutable default arguments such as `def f(x=[]):` when per-call fresh state is intended.
- `*args` → tuple; `**kwargs` → dictionary.

File Reading and Writing

Python file operations are normally performed with `open()`. The recommended pattern is a `with` statement, which ensures the file is closed when the block exits. Common text modes include `r` for reading, `w` for writing/truncating, `a` for appending and `x` for exclusive creation. `+` enables update operations, while `b` selects binary mode. `read()` reads content, `readline()` reads a line, and `readlines()` returns a list of lines. Iterating directly over a file is memory-efficient for line-oriented processing. `write()` writes a string and `writelines()` writes an iterable of strings without automatically adding separators. Text files use an encoding, often explicitly specified such as `encoding='utf-8'`. Binary files operate on bytes rather than `str`. File exceptions should be handled appropriately when files may not exist or permissions may be insufficient.

- with `open(...)` as `f` automatically closes the file.
- `w` truncates an existing file.
- `a` appends.

- Text mode uses `str`; binary mode uses `bytes`.
- Specify encoding when portability of text representation matters.

PART III — OBJECT ORIENTED PROGRAMMING AND C++

C++ Programming Features

C++ is a compiled, statically typed, general-purpose language that extends the C family with classes, object-oriented programming, templates, exceptions, references, namespaces and a large standard library. It supports procedural, object-oriented and generic programming styles. A C++ program is translated through preprocessing, compilation and linking into an executable or other target. C++ provides stronger abstraction mechanisms than C while retaining low-level capabilities such as pointers and explicit dynamic allocation. Object-oriented programming organizes software around objects that combine state and behaviour. The major concepts in this syllabus are abstraction, encapsulation, inheritance and polymorphism.

- C++ is statically typed.
- Classes define user-defined types.
- Templates provide compile-time generic programming.
- The C++ Standard Library supplies containers, algorithms, strings, streams and many utilities.

Data Abstraction

Abstraction means representing the essential characteristics of an entity while hiding unnecessary implementation details. A class can expose a public interface while keeping internal representation private. For example, a Stack class may expose push, pop and top while hiding whether it uses an array or linked structure. Users of the abstraction depend on the operations rather than the implementation. Abstraction reduces coupling: internal implementation can change while client code continues to use the stable public interface. Abstract classes, interfaces through pure virtual functions and well-designed class APIs are important C++ mechanisms for abstraction.

- Abstraction answers what an object does rather than how it internally does it.
- Private representation can be changed without changing the public contract.
- Abstract base classes support interface-oriented designs.

Encapsulation

Encapsulation combines data and the operations that manage that data and controls access through access specifiers. C++ classes provide private, protected and public members. Private members are accessible within the class and its permitted friends. Protected members are accessible within the class and derived classes, subject to the language rules. Public members form the externally accessible interface. Encapsulation protects invariants. For example, if an account balance must never become invalid, direct public modification can be prohibited and controlled member functions can enforce the invariant.

- class defaults to private access; struct defaults to public access.
- Encapsulation is stronger when mutable state is kept private.
- Access control is a compile-time language mechanism.

Class and Object

A class is a user-defined type containing data members and member functions. An object is an instance of a class with its own state. The class specifies the structure and behaviour; objects occupy storage for their non-static data members and have access to the class's functions. Member functions can access the object's state through the implicit this pointer. Static members belong to the class rather than to an individual object. Constructors initialize objects when they are created. Destructors perform cleanup when an object's lifetime ends, especially for resources managed by the class.

- Class = type definition; object = instance.
- Each non-static object generally has its own non-static data members.
- Member functions operate in the context of an object unless static.

Constructors

A constructor is a special member function used to initialize an object. It has the same name as the class and no declared return type. Constructors can be overloaded to provide different initialization choices. A default constructor can be called without arguments. A parameterized constructor accepts initialization values. A copy constructor initializes an object from another object of the same class, conventionally taking a const reference such as `T(const T&)`. Initialization lists are important because base classes and data members are initialized before the constructor body executes. A member should generally be initialized in the order of its declaration, not the order written in the list.

- Constructors do not have return types.
- Delegating constructors can reuse another constructor.
- const reference is the conventional copy-constructor parameter.
- Member initialization order follows declaration order.

Static Members

Static data members belong to the class rather than to each object. There is one shared member associated with the class, subject to the rules of the declaration and definition. Static member functions also belong to the class and can be called without an object. A static member function does not have a `this` pointer because it is not associated with a particular object. Consequently, it cannot directly access non-static members without an object through which to do so. Modern C++ permits inline static data members to be defined in the class, while older patterns commonly required a separate definition outside the class.

- Static data → shared class-level state.
- Static member function → no `this` pointer.
- Access static members using `ClassName::member`.

Constant Members

The `const` qualifier expresses that an object or member function should not modify a particular object/state through that interface. A `const` data member must be initialized, commonly through a constructor initializer list. A `const` member function is declared with `const` after its parameter list. It promises not to modify the observable non-mutable state of the object and can be called on `const` objects. `Const-correctness` allows the compiler to catch accidental modifications and communicates interface intent. A pointer and `const` can be combined in different ways: pointer to `const` prevents modification through the pointer, while `const` pointer prevents changing the pointer itself.

- `const` data members require initialization.
- `const` member function: `void f() const`.
- `const` object can call `const` member functions.
- `const int*` and `int* const` have different meanings.

Member Functions

A member function is a function declared as part of a class. It can access the class's members according to access control. It may be defined inside the class, where it is implicitly inline, or outside the class using the scope-resolution operator. Non-static member functions receive an implicit `this` pointer referring to the object on which the function was invoked. Member functions can be overloaded, `const`-qualified and, where appropriate, `virtual`. The interface of a class should expose operations that preserve its invariants. Implementation details should normally remain private or protected rather than being unnecessarily exposed.

- Outside definition uses `ClassName::functionName`.
- Non-static member functions have `this`.
- `const`-qualified member functions can operate on `const` objects.

Pointers and References in C++

A pointer stores an address and can be null, reassigned and used with pointer arithmetic when pointing into an appropriate array/object. A reference is an alias for an existing object and is normally required to refer to a valid object when initialized. References cannot be reseated to refer to another object. References are commonly used for function parameters to avoid copying and, when non-const, to allow modification of the referred object. const references can bind to temporary values and are widely used for efficient read-only parameter passing. Pointers are useful when optional ownership, nullable state, dynamic allocation or pointer-based data structures are required. References are often preferable when the function requires an existing object and null is not a meaningful state.

- Pointer → object containing an address-like value; reference → alias.
- Reference must be initialized.
- Pointer can be reassigned; reference cannot be reseated.
- Use const reference for efficient read-only access to potentially large objects.

Role of the this Pointer

In a non-static member function, this is a pointer to the object for which the function was called. It allows the function to distinguish members from parameters with the same name and can be used to return *this for fluent interfaces. For example, a constructor parameter named x can initialize a member x using this->x = x, although a member initializer list is generally preferred for initialization. Because static member functions are not invoked for a particular object, they do not have a this pointer.

- this refers to the current object.
- this->member accesses a member through the current object's pointer.
- Returning *this can support chaining.
- No this in static member functions.

Storage Classes / Storage Duration in C++

C++ separates several related concepts: storage duration, linkage and scope. Automatic objects commonly have automatic storage duration; static objects have static storage duration; dynamically allocated objects have dynamic storage duration. The static keyword can give a local variable persistent lifetime or give a namespace/file-level entity internal linkage depending on context. extern can declare an entity defined elsewhere. Thread-local storage can be requested with thread_local. Modern C++ design often avoids manual storage-duration management by using RAII: an object's lifetime is tied to scope, and its destructor releases resources automatically. This principle is central to safe C++ resource management.

- Automatic → typically scope-based lifetime.
- Static → program-duration storage.
- Dynamic → allocated until explicitly released or ownership mechanism destroys it.
- RAII ties resource lifetime to object lifetime.

Functions as Arguments

C++ allows functions to be passed or represented as arguments through function pointers, function objects and lambda expressions. A function pointer stores the address of a function with a compatible signature. The standard library also provides callable abstractions such as std::function. Function templates can accept generic callables. Lambdas provide concise unnamed function objects and can capture variables from surrounding scopes. Passing functions enables higher-order algorithms such as sorting with a custom comparison predicate, filtering, transformations and callbacks.

- Function pointer syntax includes the pointed-to function's parameter and return types.
- Lambda expressions create callable objects.

- `std::function` provides a type-erased callable wrapper.
- Standard algorithms commonly accept callable predicates/comparators.

String Handling in C++

C++ supports C-style character arrays and the safer, richer `std::string` class from the standard library. `std::string` is a dynamic sequence of characters and manages its own storage. It supports concatenation, comparison, searching, substring extraction and iteration. C++ strings can be constructed from literals and manipulated through member functions such as `size`, `length`, `substr`, `find`, `append` and `replace`. `operator+` and `operator+=` support concatenation. `std::getline` reads a complete line from an input stream, including spaces until the delimiter. `std::string` should generally be preferred for ordinary text because it manages memory automatically. `c_str()` provides access to a null-terminated character representation when interfacing with C APIs, subject to the lifetime rules of the string object.

- `std::string` is mutable.
- `size()` and `length()` return the number of characters.
- `find()` returns a position or `std::string::npos` when not found.
- `std::getline` handles spaces in line input.

Copy Constructor

A copy constructor creates an object from another object of the same class. The usual form is `Class(const Class& other)`. If a class owns a resource through a raw pointer, a compiler-generated memberwise copy may copy only the pointer value, producing two objects referring to the same resource. This is called a shallow copy. If independent ownership is required, a deep copy may allocate separate storage and copy the pointed-to contents. Modern C++ often prefers RAII types such as `std::string`, `std::vector` and smart pointers, which reduce the need for manually written deep-copy logic. Copy constructors are one component of the broader rule of three/five/zero. A resource-owning class may need appropriate copy/move constructors and assignment operators, or it can use types that manage resources safely.

- Copy constructor initializes a new object.
- Shallow copy duplicates pointer values.
- Deep copy duplicates owned resources where independent ownership is intended.
- Prefer RAII standard-library types over raw owning pointers.

Polymorphism

Polymorphism means that one interface can represent or operate on objects of different forms. In C++, compile-time polymorphism includes function overloading, operator overloading and templates. Runtime polymorphism is primarily achieved through virtual functions and inheritance. With a base-class pointer or reference referring to a derived object, a virtual member function call can dispatch to the derived override at runtime. This allows code to work against an abstract interface while concrete derived classes provide different implementations.

- Compile-time polymorphism → resolved by compilation/type information.
- Runtime polymorphism → dynamic dispatch through virtual functions.
- Use `override` in derived classes to let the compiler verify overriding intent.

Function Overloading

Function overloading allows multiple functions in the same scope to have the same name but different parameter lists. The compiler selects the best viable overload based on the arguments. Return type alone cannot distinguish overloaded functions. Overloading improves expressiveness when operations are conceptually the same but accept different types or numbers of arguments. Ambiguous conversions can make overload resolution fail, so implicit conversions should be considered carefully.

- Parameter type/number/order can distinguish overloads.

- Return type alone cannot.
- Overload resolution is compile-time.

Operator Overloading

C++ permits many operators to be overloaded for user-defined types. An overloaded operator is a function whose syntax allows an object-oriented interpretation of an operator such as `+`, `==` or `<<`. Operator overloading should preserve intuitive meaning. It cannot create entirely new operators, change an operator's precedence, or alter the number of operands. Some operators have restrictions on how they may be overloaded, and at least one operand of an overloaded operator must involve a user-defined type under the language rules. Stream insertion and extraction are commonly overloaded for custom classes using `operator<<` and `operator>>`.

- Operator overloading does not change precedence.
- It does not invent new operator symbols.
- Keep overloaded operators semantically natural.
- Some operators cannot be overloaded, such as `::`, `.` and `.*` in standard C++.

Dynamic Memory Allocation

Dynamic memory is obtained during program execution. In C++, `new` allocates an object and returns a pointer to it; `delete` releases an object allocated with `new`. `new[]` allocates an array and `delete[]` releases an array allocated with `new[]`. Incorrect pairing, leaks, double deletion and dangling pointers are common hazards. Modern C++ strongly encourages smart pointers such as `std::unique_ptr` and `std::shared_ptr` for ownership management. `unique_ptr` represents exclusive ownership and is inexpensive; `shared_ptr` provides shared ownership using reference counting. RAII is preferable to manually matching every allocation and deallocation because ownership is represented by object lifetime and cleanup occurs automatically through destructors.

- `new` ↔ `delete`.
- `new[]` ↔ `delete[]`.
- Dangling pointer → pointer referring to an object whose lifetime ended.
- Memory leak → allocated resource becomes unreachable without being released.
- `unique_ptr` is the default smart-pointer choice for exclusive ownership.

Nested Classes

A class can be declared inside another class. The nested class is a member of the enclosing class and provides a way to group a helper type closely related to the outer abstraction. The nested class has its own scope and access rules. Nesting is primarily an organizational mechanism and can make implementation-specific helper types less visible to unrelated code.

- Nested class is a member of the enclosing class.
- It has its own type and scope.
- Nesting can express a strong conceptual relationship between helper and enclosing types.

Inheritance

Inheritance allows a derived class to extend or specialize a base class. Public inheritance is commonly used to model an 'is-a' relationship and preserve substitutability: a derived object can be used where a base object is expected, subject to the interface contract. C++ supports single, multiple, multilevel and hierarchical inheritance. Virtual inheritance addresses a particular diamond-inheritance issue by allowing a shared virtual base subobject. Base constructors execute before derived constructors, and derived destructors execute before base destructors. Access control and inheritance mode determine how inherited members are exposed through the derived class.

- Base constructor runs before derived constructor.

- Derived destructor runs before base destructor.
- Public inheritance generally models substitutability.
- Multiple inheritance permits more than one direct base.
- Virtual inheritance can prevent duplicate base subobjects in a diamond.

Virtual Functions

A virtual function enables dynamic dispatch when called through a base pointer or reference. If a derived class overrides the virtual function, the derived implementation can be selected at runtime according to the actual object's dynamic type. A pure virtual function is declared with `=0` and makes a class abstract when the class has at least one pure virtual function. A virtual destructor is important for a polymorphic base class when derived objects may be deleted through base pointers; it ensures the derived destructor is invoked. The override specifier is strongly recommended for intended overrides because it lets the compiler detect signature mismatches.

- virtual enables runtime dispatch.
- pure virtual function → interface-like requirement.
- Abstract class cannot normally be instantiated.
- Polymorphic base classes generally need virtual destructors when deletion through base pointers is possible.

Abstract Class

An abstract class contains at least one pure virtual function and cannot be instantiated directly. It can define common data and implemented functions while requiring derived classes to provide specific behaviour. Abstract classes are useful as interfaces or partial implementations. A pointer or reference to an abstract base can refer to a concrete derived object, enabling runtime polymorphism.

- Cannot instantiate an abstract class.
- Can have constructors, data members and non-pure member functions.
- Derived concrete class must implement required pure virtual functions.

Exception Handling

C++ exception handling separates error detection from error recovery. A function can throw an exception using `throw`. A try block contains code whose exceptions may be handled, and catch clauses specify handlers. Exceptions are objects or values whose type can communicate the nature of the failure. Catching by const reference is often preferred for class-type exceptions to avoid unnecessary copying and slicing. Stack unwinding destroys automatic objects as control leaves scopes, which is one reason RAII works naturally with exceptions. Standard exception classes such as `std::exception` and derived types are provided by the library. Exceptions should represent exceptional failure rather than ordinary control flow.

- `throw` transfers control to a matching handler.
- catch handlers are considered for compatible exception types.
- Stack unwinding destroys automatic objects as scopes are exited.
- RAII prevents resource leaks during exception propagation.

Standard Libraries

The C++ Standard Library provides reusable components rather than requiring programmers to implement common data structures and algorithms manually. Important facilities include containers in `<vector>`, `<list>`, `<map>`, `<set>`; algorithms in `<algorithm>`; strings in `<string>`; iterators; utilities; smart pointers; streams; exceptions; and numeric facilities. The standard library is heavily template-based. This permits generic algorithms to operate on many container and user-defined types while preserving compile-time type checking. Using the standard library generally improves correctness, maintainability and development speed because well-tested components can replace custom implementations.

- `std::vector` is a dynamic contiguous sequence.
- `std::list` is a doubly linked list.
- `std::map` is an ordered associative container.
- `std::unordered_map` is a hash-based associative container.
- `std::algorithm` contains many generic algorithms.

Generic Programming

Generic programming develops algorithms and data structures independently of a particular concrete type. C++ templates are the principal language mechanism for generic programming. A generic function can operate on any type satisfying the operations used by the function. A generic class can parameterize the type of its stored elements or policies controlling its behaviour. Templates are instantiated for the required types, giving compile-time type checking and often allowing optimization through specialization and inlining. Modern C++ also uses concepts to express template requirements explicitly, although the core syllabus focus is templates.

- Generic code is parameterized by type or value.
- Templates are primarily compile-time mechanisms.
- STL containers and algorithms are template-based.

Function Templates

A function template defines a family of functions parameterized by one or more template parameters. For example, a `max`-like function can compare two values without writing separate functions for `int`, `double` and other comparable types. Template argument deduction often determines template parameters from function arguments. Explicit template arguments can also be supplied. If the required operations are not available for a type, compilation fails unless another overload or specialization applies.

- template declares a type parameter.
- Type can often be deduced from function arguments.
- Instantiation occurs for the types used by the program.

Class Templates

A class template defines a family of classes parameterized by types or values. `std::vector` is a classic example: `vector` and `vector<int>` are different specializations of the same class template pattern. Class templates may have multiple parameters, default template arguments and specialized implementations. A class template can expose member functions that operate on the parameterized type.

- `vector` and `vector<int>` are different specializations.
- Templates permit reusable type-independent data structures.
- Template parameters may represent types or compile-time values.

STL — Standard Template Library

The term STL is commonly used for the collection of C++ generic containers, iterators and algorithms. Its design separates data structures from algorithms through iterator-based interfaces. Containers hold data. Iterators provide a generalized mechanism for traversing container elements. Algorithms such as `sort`, `find`, `copy`, `transform` and `accumulate` operate on iterator ranges rather than requiring one algorithm per container type. This separation is powerful because the same algorithm can work with many containers as long as their iterators satisfy the algorithm's requirements.

- Container + iterator range + algorithm is a central STL pattern.
- Algorithms are generally in the `std` namespace.
- Iterator categories determine which operations an algorithm can efficiently perform.

STL Containers

Sequence containers include `vector`, `deque`, `list`, `forward_list` and `array`. Associative containers include `set`, `multiset`, `map` and `multimap`, generally organized around ordered keys. Unordered associative containers include `unordered_set`, `unordered_multiset`, `unordered_map` and `unordered_multimap`, using hashing. `vector` offers contiguous storage and fast random access. `deque` supports efficient insertion/removal at both ends with random access. `list` supports constant-time insertion/removal when the iterator position is known but does not support random access. `map` stores key-value pairs ordered by key, while `unordered_map` uses hashing and provides expected constant-time lookup. Container choice should follow required operations rather than habit.

- `vector` → contiguous, random access.
- `list` → linked, no random access.
- `deque` → efficient both-end operations.
- `set/map` → ordered associative containers.
- `unordered_set/unordered_map` → hash-based associative containers.

STL Iterators

Iterators generalize pointers and provide traversal through container elements. Different iterator categories support different capabilities: input, output, forward, bidirectional and random-access iterators; modern C++ additionally recognizes contiguous iterators. A `vector` iterator supports random movement and arithmetic. A `list` iterator is bidirectional but does not support random jumps such as `it+n`. Algorithms specify the minimum iterator category they require. `begin()` and `end()` define a half-open range `[begin,end)`, where `begin` refers to the first element and `end` is one position past the last element. Reverse iterators provide reverse traversal through `rbegin()` and `rend()`.

- `end()` is one-past-the-end and must not be dereferenced.
- Half-open ranges are fundamental to STL algorithms.
- Iterator invalidation rules depend on the container and operation.

Function Adaptors

Function adaptors are utilities that modify or wrap callable objects so that they can be used conveniently with algorithms. Older STL terminology includes binders and negators; modern C++ commonly uses lambdas, `std::bind` and function objects. `std::bind` creates a callable object with selected arguments bound to specified values or placeholders. Lambdas are usually clearer for new code because they directly express the intended operation and can capture surrounding values. Other functional utilities include `std::function` for storing callables and `std::mem_fn` for adapting member functions.

- `std::bind` creates a callable with bound arguments.
- Lambdas are the modern, readable alternative for many adaptor use cases.
- `std::function` type-erases compatible callables.

Allocators

An allocator abstracts memory allocation for standard-library containers. The classic allocator model separates obtaining raw storage from constructing objects in that storage. Containers use allocator-aware mechanisms so memory management can be customized when necessary. `std::allocator` is the standard default allocator type. Custom allocators can be useful in specialized systems, memory pools, shared-memory environments or performance-sensitive applications, although ordinary application code rarely needs to write one.

- Allocator manages raw storage for container elements.
- Allocator customization is mainly a specialized facility.
- Container memory management and object lifetime are related but distinct concepts.

Parameterizing the Class

Parameterizing a class means designing the class so that one or more characteristics are supplied as template parameters rather than hard-coded. A simple container template can take an element type `T`. More advanced templates can take comparison policies, allocators, sizes or other compile-time parameters. For example, a generic stack can store any suitable type. A policy parameter can allow the user to choose how elements are compared or stored without rewriting the class. This is a central generic-programming technique.

- Type parameter → element type or other type dependency.
- Non-type template parameter → compile-time value.
- Policy/template parameters can customize behaviour while retaining compile-time checking.

File Handling Concepts in C++

C++ performs file I/O primarily through stream classes such as `std::ifstream` for input, `std::ofstream` for output and `std::fstream` for both. The stream abstraction treats files similarly to other C++ I/O sources and destinations. Text output uses `operator<<` and input uses `operator>>`. `std::getline` reads complete lines. Binary I/O can use `read()` and `write()` with character buffers. Opening a stream should be checked for success, and RAII automatically closes the stream when its object goes out of scope. File positions can be controlled using `seekg/tellg` for input and `seekp/tellp` for output. Binary serialization of arbitrary objects is not automatically portable because object representation, padding and pointer values are implementation-dependent.

- `ifstream` → file input.
- `ofstream` → file output.
- `fstream` → both.
- RAII closes file streams when their objects are destroyed.
- Use `std::ios::binary` for binary mode when appropriate.

REFERENCE CONCEPT MAP

C programming: types → expressions → I/O → decisions → loops → arrays/strings → algorithms → functions → recursion → pointers → structures/unions → storage/preprocessor → files.

Python: interpreter/REPL → objects and data types → statements/expressions → Boolean logic/operators → strings → numeric arrays → lists/tuples/dictionaries → functions → files.

C++ / OOP: language features → abstraction → encapsulation → class/object → constructors → static/const/member functions → pointers/references/this → functions as arguments → strings → copy construction → polymorphism → overloading → dynamic memory → nested classes → inheritance → virtual/abstract classes → exceptions → standard library → generic programming → templates → STL → file handling.

IMPORTANT CODE PATTERNS

```
/* C: pointer used to modify caller's value */
void change(int *p) {
    *p = 100;
}
int x = 5;
change(&x);

/* C: structure and pointer */
struct Node {
    int data;
    struct Node *next;
};

/* C: dynamic memory */
int *p = malloc(n * sizeof *p);
if (p != NULL) {
    /* use p */
}
free(p);

# Python: list, tuple and dictionary
numbers = [10, 20, 30]
point = (4, 7)
student = {"name": "A", "mark": 90}

# Function
def add(a, b):
    return a + b

# File
with open("data.txt", "r", encoding="utf-8") as f:
    text = f.read()

// C++: class, constructor and const member function
class Account {
    int balance;
public:
    Account(int b) : balance(b) {}
    int getBalance() const { return balance; }
};

// Runtime polymorphism
class Base {
public:
    virtual void show() const = 0;
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void show() const override {}
};
```

The code patterns above are representative constructions. The detailed explanations earlier in the document describe the language rules behind them; the examples are not intended as a substitute for understanding those rules.